SoftSic: Gesamtpräsentation

Vilrose Daquiado

Fahrrad Tachometer Anforderungen

Anforderungen

Funktionale Anforderungen:

- Erfassung von Radumdrehungen über Rechtecksignal (z. B. Hall-Sensor)
- Berechnung von Geschwindigkeit und Strecke
- Datenübertragung in die Cloud (z. B. per WLAN, LTE, Bluetooth)

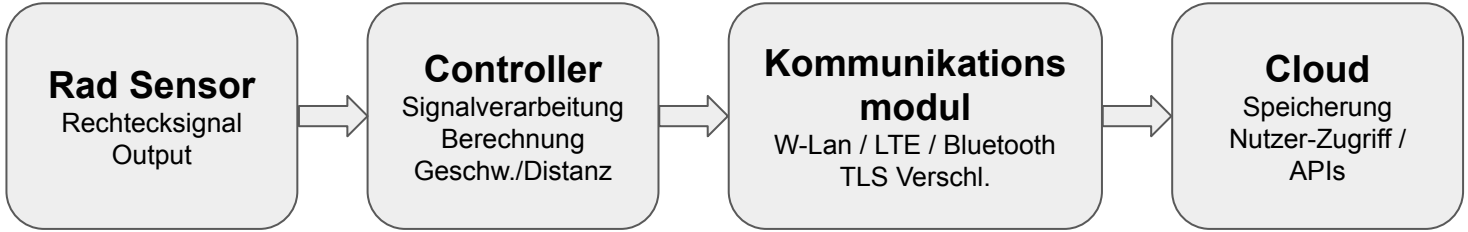
Nicht-funktionale Anforderungen:

- Echtzeitverarbeitung für präzise Geschwindigkeitsmessung
- Hohe Verfügbarkeit & Ausfallsicherheit
- Geringer Energieverbrauch (z. B. Akku-betriebenes Gerät)

Sicherheitsanforderungen:

- Vertraulichkeit & Integrität der übertragenen Daten
- Schutz vor Manipulation der Messdaten
- Authentifizierung bei Cloud-Kommunikation
- Sichere Firmware-Updates (OTA)

Architektur



Mögliche Angriffe

Angriffsfläche	Mögliche Angriffe	Mögliche Schutzmaßnahmen
Sensor	Manipulation (Magneten, Jammer)	Plausibilitätsprüfungen
MCU (Controller)	Firmware-Manipulation, Debug-Port-Zugriff	Secure Boot, Port Lockdown
Funkverbindung	MITM, Spoofing	TLS, Authentifizierung
Cloud	Unbefugter Zugriff, Datenlecks	Zugriffskontrollen, Verschlüsselung
OTA-Updates	Gefälschte Firmware	Digitale Signaturen, Prüfmechanismen

Stuxnet Attack Tree Aufgabe

Stuxnet

Ziel:
Sabotage eines Industrial Control Systms (ICS) durch Veränderung von PLC-Code

1. Informationsbeschaffung
    - 1.1. Zugang zu ICS-Spezifikationen
        - 1.1.1. Insider stiehlt ICS-Dokumentation
        - 1.1.2. Frühere Malware zur Datenextraktion
    - 1.2. Aufbau einer Testumgebung
        - 1.2.1. Nachbau der Ziel-ICS-Umgebung
        - 1.2.2. Validierung der Payload gegen reale Hardware
    - 1.3. Code-Signierung
        - 1.3.1. Diebstahl von digitalen Zertifikaten
        - 1.3.2. Signierung von Treibern zur Umgehung von Sicherheitssoftware
3. Initiale Infektion
    - 3.1. USB-Infektion eines internen Mitarbeiters / Lieferanten (wissentlich oder unwissentlich)
4. Lokale Ausbreitung im Zielnetzwerk
    - 4.1. Verbreitung über Netzwerk (LAN)
    - 4.2. Verbreitung über USB
5. Zielsystemkompromittierung
    - 5.1. Suche nach Siemens Step7-Software
    - 5.2. PLC-Infektion
        - 5.2.1. Einschleusung und Änderung von Code
        - 5.2.2. PLC-Rootkit zur Verbergen der Änderungen
6. Command & Control
    - 6.1. Update-Fähigkeit
        - 6.1.1. Peer-to-peer Verbreitung
    - 6.2. Datenextraktion

Stuxnet - Attack Tree

Sichere Passwort-Eingabe

Sichere Passwort-Eingabe

- in C-Pseudocode
- 16MB RAM / 16 MB Persistent
- PW muss sicher abgelegt werden (AttackSpace Minimierung)
- PW beliebig lang
- Bei Überprüfung kein Bufferoverflow

```
// --- Konstanten ---
DEFINE BLOCK_SIZE = 8 * 1024 * 1024  // 8 MB
DEFINE HASH_SIZE = 32                // z. B. SHA-256 = 32 Byte

// --- Speicherbereiche ---
DECLARE BYTE char_array[BLOCK_SIZE]
DECLARE INTEGER char_index = 0
DECLARE BYTE hash[HASH_SIZE] = {0}

FUNCTION HASH_BLOCK:
   BYTE h[HASH_SIZE]
   HASH(h, char_array, char_index)
   HASH(hash, CONCAT(hash, h), 2 * HASH_SIZE)
   FOR (int i = 0; i < BLOCK_SIZE; ++i):
      char_array[i] = 0
END FUNCTION
```

```
// --- Hauptschleife ---
LOOP:
   CHAR c = WARTE_AUF_TASTENDRUCK()
   IF c == EINGABE_ENDE
      IF char_index > 0 THEN
         FOR (char_index; char_index < BLOCK_SIZE; ++char_index):
            char_array[char_index] = 0
         CALL HASH_BLOCK
      END IF
      BREAK
   END IF
   // Zeichen speichern
   char_array[char_index] = c
   char_index = char_index + 1

   // Puffer ist voll – blockweise hashen
   IF char_index == BLOCK_SIZE THEN
      CALL HASH_BLOCK
      char_index = 0
   END IF
END LOOP

// --- Weiterverarbeitung ---
VERARBEITE(hash)
```

Fahrrad Tacho Use & Misuse Case Diagramme

Sicherheitsziele für zentrale Assets

Asset	Sicherheitsziele
Sensordaten	Integrität, Verfügbarkeit
Nutzerdaten	Vertraulichkeit, Integrität
Kommunikationsschnittstelle(Bluetooth, WLAN)	Vertraulichkeit, Integrität, Authentizität
Firmware	Integrität, Authentizität
Cloud-Datenbank	Vertraulichkeit, Integrität, Verfügbarkeit
Tacho- Hardware (Speicher, Anzeige)	Verfügbarkeit, Integrität

Bedrohungen je Asset (STRIDE-orientiert)

STRID-Kategorie	Bedrohung
Spoofing	Falsche Geräte geben sich als Nutzer aus
Tampering	Angreifer verändert Firmware oder Sensorwerte
Repudiation	Nutzung ohne Protokollierung
Information Disclosure	Bluetooth sniffing, Cloud-Datenlecks
DoS	Akku-Drain-Angriffe, Sensor-Störung
Elevation of Privilege	OTA-Hack mit privilegierter Firmware

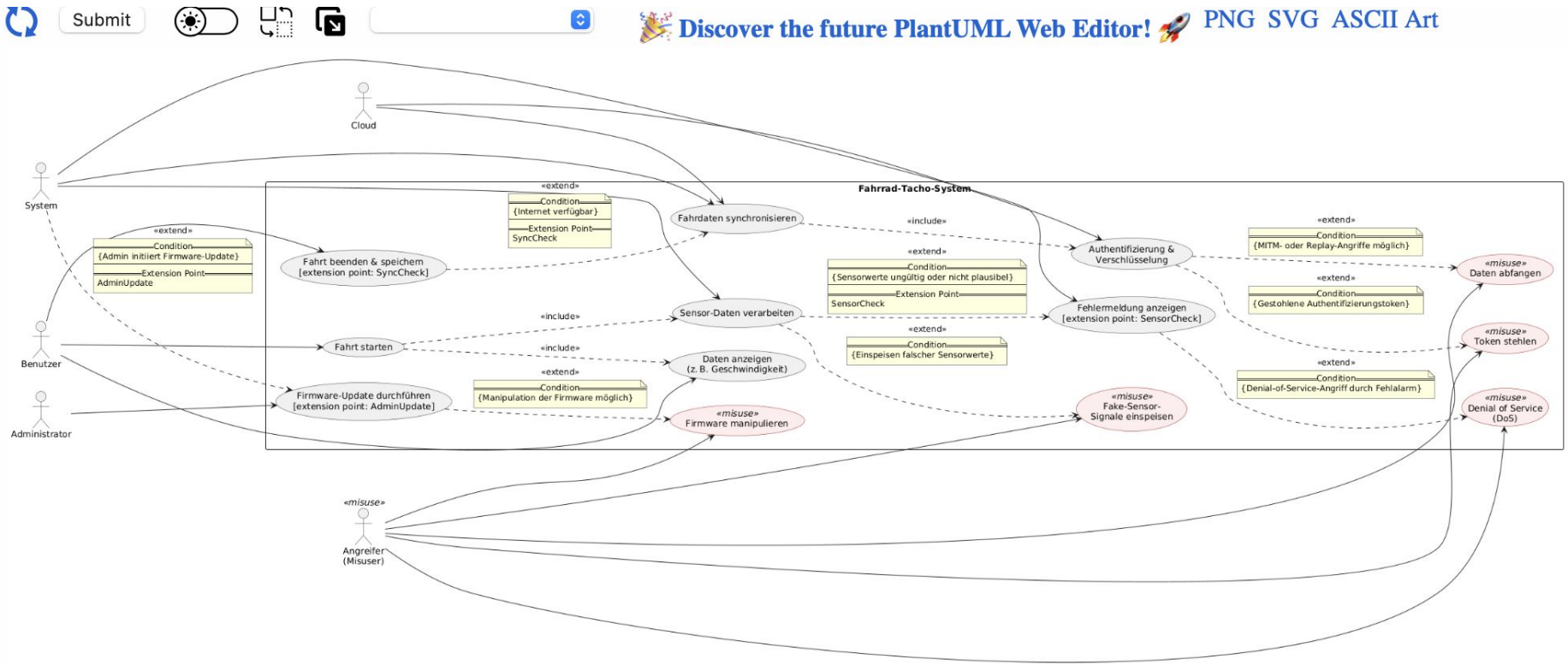
Risiken (Kombination aus Bedrohung + Schadenshöhe + Eintrittswahrscheinlichkeit)

Risiko	Einschätzung (hoch/mittel/Niedrig)	Begründung
Manipulierte Sensor-daten führen zu fehlerhafter Anzeige	Mittel	Gefahr für Nutzer (z.B. Geschwindigkeit unterschätz)
Firmware wird durch Schadsoftware ersetzt	Hoch	Komplettte Systemübernahmen möglich
Daten werden beim Sync abgefangen (z.B. MITM)	Hoch	Komplette Systemübernahme möglich
Verfügbarkeit durch DoS über Sensorüberlatung	Niedrig	Spezialwissen nötig
Nutzerprofil wird aus der Cloud gestohlen	Hoch	Persönliche Daaten kompromiert

Sicherheitsanforderungen aus den identifizierten Risiken

Risiko	Sicherheitsanforderungen
Manipulierte Sensordaten	Sensorwerten digital signieren oder auf Plausibilität prüfen
Firmware Manipulation	Firmware-Signatur prüfen, Secure Boot verwenden
Datenabgriff beim Sync	Ende-zu-Ende-Verschlüsselung (z.B. TLS), Authentifizierung über Token
DoS über Sensorüberlastung	Eingangsrate drosseln, Anomalie-Erkennung implementieren
Diebstahl von Cloud-Daten	Zugriff per Authentifizierung (OAuth2), Verschlüsselung ruhender Daten

Use & Misuse Case Diagramm



Link zum Diagramm

strcpy-Routine

Übung in Sichere Software

Aufgabe

Anforderungen:

- strcpy routine
- beliebig lange strings
- schutz gegen buffer overflow
- funktionssignatur: char* strcpy(char* src, char* dest)

Lösung

```
char* strcpy(char* dest, char* src) {
    if (src == NULL) { return NULL; }
    size_t src_len = strlen(src);
    if (dest == NULL) {
        dest = malloc(src_len + 1);
        if (dest == NULL) { return NULL; }
    }
    for (size_t i = 0; i <= src_len; ++i) {
            dest[i] = src[i];  // inkl. '\0'
        }
    return dest;
}
```

Problem: dest nicht null, aber zu klein

Lösung:

```
char* safe_strcpy(char* dest, size_t
dest_size, const char* src) {
    if (dest == NULL || src == NULL
            || dest_size == 0) {
        return NULL;
    }
    size_t src_len = strlen(src);
    if (src_len >= dest_size) {
        strncpy(dest, src, dest_size - 1);
        dest[dest_size - 1] = '\0';
        return NULL;
    }
    return strcpy(dest, src);
}
```

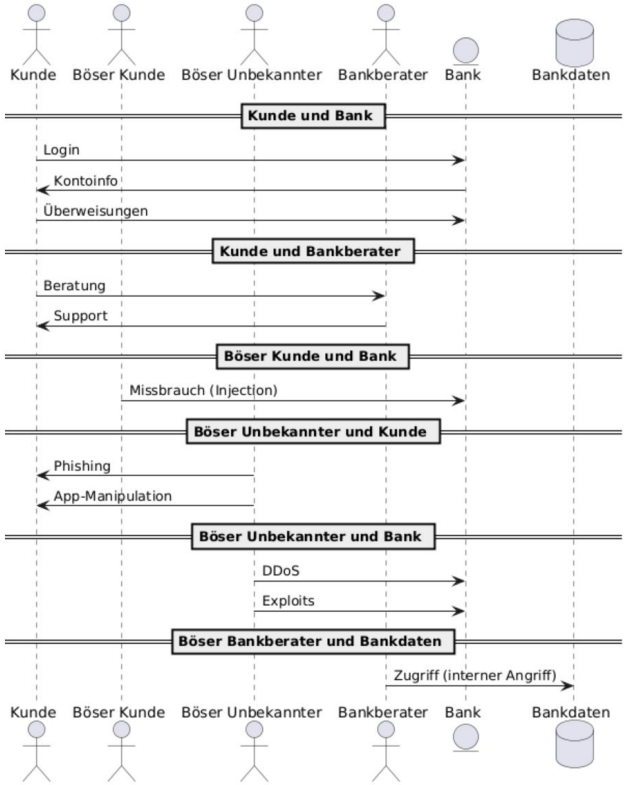
BANKING APP SHOWCASE

Nicolas Meyer & Vilrose Daquiado

ANFORDERUNGEN

FUNKTIONALE ANFORDERUNGEN	NON-FUNKTIONALE ANFORDERUNGEN
[hoch] Überweisungen tätigen	Erfüllung von Compliance Anforderungen • Datenschutz • Privacy by Design • Erfüllung der Bankenanforderungen ○ BA/VAIT ○ EU Zahlungsrichtlinie
[mittel] [Compliance: Hoch] Kontoinformationen abrufen Kontostand • Kontoauszüge • Kontodaten (Inhaber und IBAN) • Postfach	Erfüllung der Sicherheitsanforderungen • Starke Authentifikation des User • Vertrauliche und sichere Kommunikation mit der Bank • Authentisierung der Bank • Schutz der lokalen Daten
[hoch] Kunden Selfservice	

1.1.1 AUFGABE: Kommunikationsflüsse



Link zum PlantUML

1.1.2a Risiken bei der Benutzung der App

Risk	Asset	Schwachstelle	Bedrohung
Hoch	Zugangsdaten (PIN/TAN)	Diebstahl von Zugangsdaten. kein Schutz gegen Phishing, schwaches Passwort	Phishing, Brute-Force, Dictionary Attacks
Hoch	Kundenkonto	Kein 2FA, unsichere oder nicht zeitlich begrenzte Sessions. Konto-Übernahmen	Session Hijacking
Hoch	Transaktionsdaten	Unverschlüsselte Kommunikation. Verlust sensible Daten	Datenleck oder MITM

1.1.2b Risiken bei der Bereitstellung der App

Risk	Asset	Schwachstelle	Bedrohung
Hoch	IT-Infrastruktur	Ausfall von Systemen. DDoS-Schutz fehlt, keine Redundanz	Böser Unbekannter legt Server lahm
Hoch	Kundendatenbank	Datenverlust/-diebstahl. Fehlende Verschlüsselung/Backups	Externer oder interner Angriff
Hoch	API und Backend	Kompromittierte Kommunikation. Keine TLS/SSL-Absicherung	MITM

1.1.3a SOFTWARESEITIGER MAßNAHMEN FÜR DIE TOP 3 RISIKEN

Risiko: Kompromittierte Kommunikation (z. B. MitM-Angriffe)

Asset: API & Backend
Schwachstelle: Fehlende TLS-Absicherung, keine Serverauthentizität
Bedrohung: Abhören oder Manipulation von API-Kommunikation

Maßnahme:

Die gesamte Kommunikation zwischen App, API und Backend erfolgt ausschließlich über TLS 1.3, ergänzt durch Zertifikatspinning zur Vermeidung von manipulierten Zertifikaten, und alle internen Services erfordern mTLS-Authentifizierung.

1.1.3b SOFTWARESEITIGER MAßNAHMEN FÜR DIE TOP 3 RISIKEN

Risiko: Datenverlust oder -diebstahl (z. B. durch Insider oder Angreifer)

Asset: Kundendatenbank
Schwachstelle: Fehlende Verschlüsselung, keine Backups
Bedrohung: Externe oder interne Angriffe auf gespeicherte Daten

Maßnahme:

Alle sensiblen Daten in der Kundendatenbank werden mittels starker Verschlüsselung (z. B. AES-256) gespeichert, der Zugriff erfolgt nur rollenbasiert (RBAC), und es wird ein automatisiertes Backup-System mit Integritätsprüfung eingerichtet

1.1.3c SOFTWARESEITIGER MAßNAHMEN FÜR DIE TOP 3 RISIKEN

Risiko: Ausfall von Systemen

Asset: IT-Infrastruktur
Schwachstelle: Fehlende DDoS-Schutz, keine Redundanz
Bedrohung: Serverüberlastung, Nichtverfügbarkeit der App

Maßnahme:

Zur Minimierung des Risikos wird ein DDoS-Schutz auf Infrastrukturebene (z. B. Reverse Proxy mit Rate-Limiting) implementiert, zusätzlich erfolgt die Bereitstellung über eine skalierbare Cloud-Infrastruktur mit automatischem Failover und Lastverteilung (Load Balancer).

1.1.4 SOFTWARE DESIGN FÜR DIE APP INKLUSIV MAßNAHMEN

Presentation Layer (Client)

-Loginmaske mit 2FA (z.B. TOTP)
-Überweisungsformular mit Validierung
-TLS 1.3 gesicherte Kommunikation zur API
-Device Binding, Session Token Storage

→

Application / API Layer (Gateway API)

-API Gatewas mit mTLS + Zertifikatspinning
-DDos-Schutz, Rate-Limiting, IP-Filter
-Zugriffskontrolle

→

Business Logic Layer (Services)

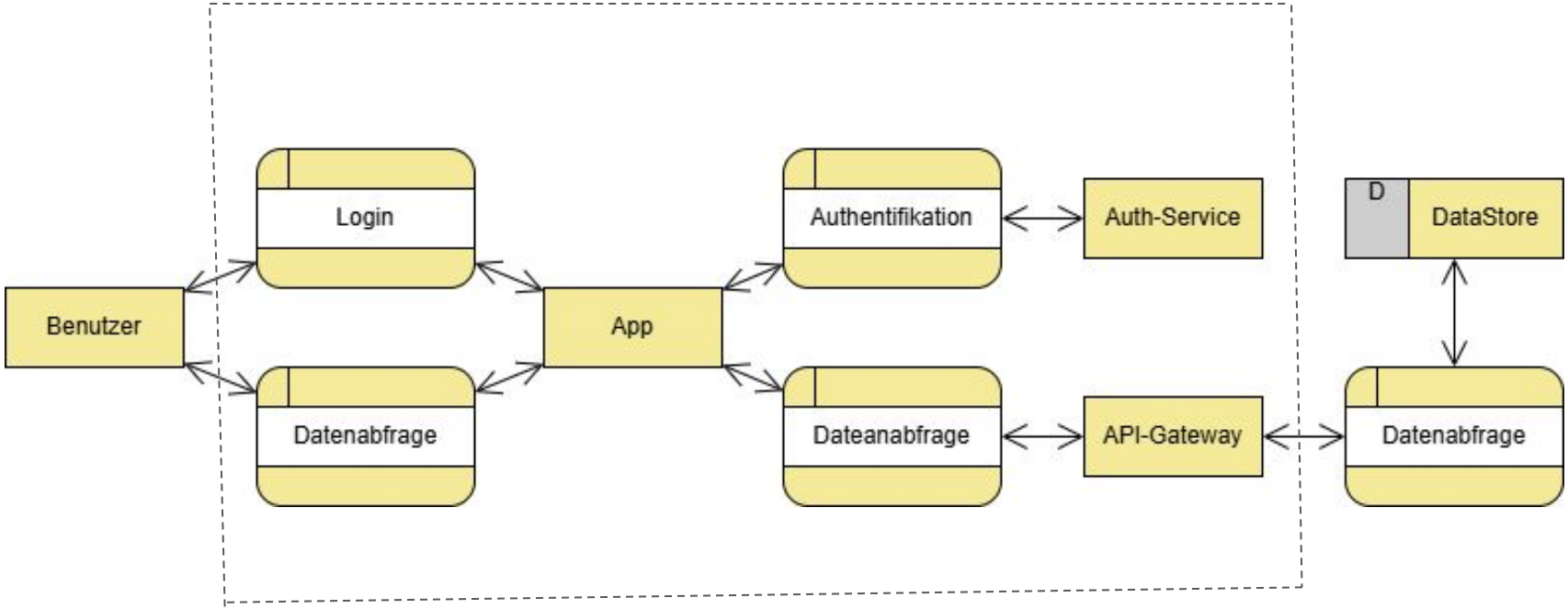
-Auth-Sevice(2FA, Login Härtung)
-Transaction-Service(Überweisung prüfen + signieren)
-Customer-Service (Kontoübersicht, Postfach)
-Logging + Anomalieerkennung

→

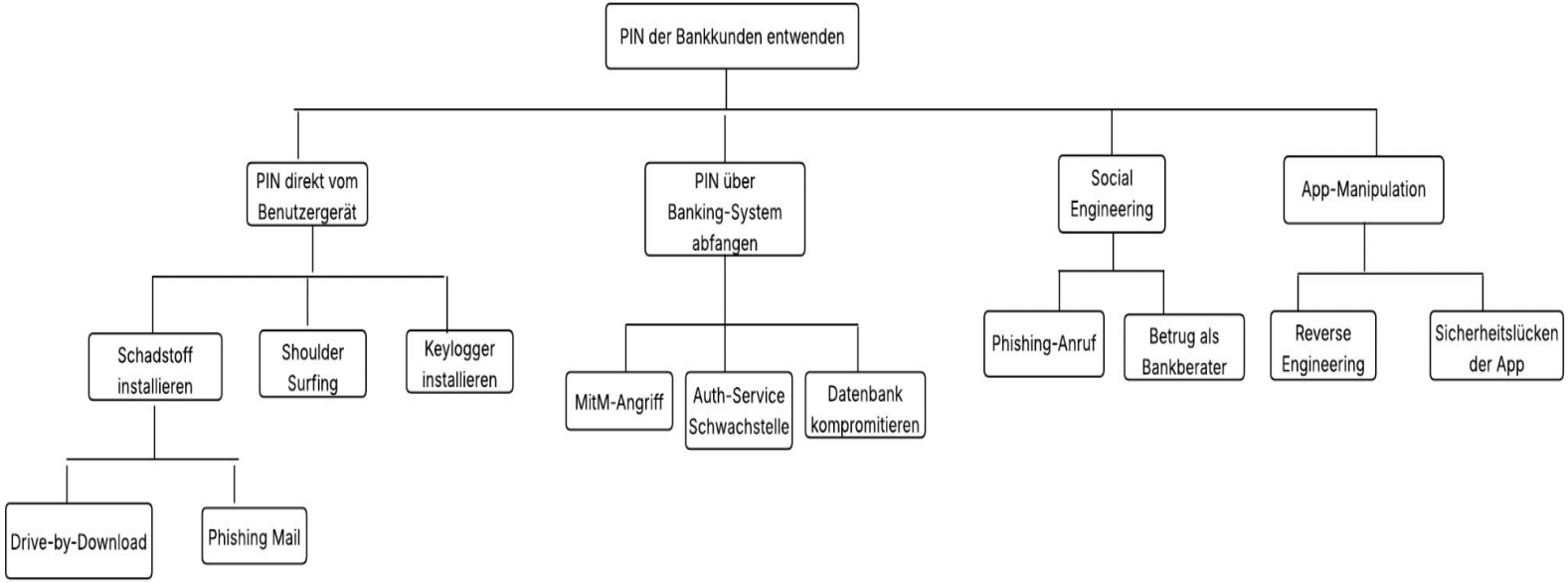
Data Layer (Datenbank)

-Kundendaten verschlüsselt (AES-256)
-Getrennte Datenbanken (Benutzerdaten, Transaktionen, Logs)
-Automatisierte Backups mit Integritätsprüfung

1.1.5 DATENFLUSSDIAGRAM



1.2 ATTACK TREES



2 TECHNISCHE AUFGABEN

2.1 ZUFALLSZAHLEN

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdint.h>
#include <errno.h>
#include <limits.h>

int main() {
    const int MIN = 1;
    const int MAX = 1000;
    const uint32_t RANGE = MAX - MIN + 1;

    uint32_t random_value;
    //Vermeidung von Bias bei der Modulo-Operation
    uint32_t max_acceptable = UINT32_MAX - (UINT32_MAX % RANGE);
        ///dev/urandom ist ein Pseudo-Zufallsgenerator (CSPRNG) (MacOS)
    int fd = open("/dev/urandom", O_RDONLY);
    if (fd < 0) {
        perror("Kann /dev/urandom nicht öffnen");
        return 1;
    }

    // Schleife, um Modulo-Bias zu vermeiden
    do {
        ssize_t r;
        //Wiederholt read(), falls es durch ein Signal (EINTR) unterbrochen wird
        do {
            r = read(fd, &random_value, sizeof(random_value));
        } while (r == -1 && errno == EINTR); // Wiederholen bei unterbrochenem Systemauf

        if (r != sizeof(random_value)) {
            perror("Fehler beim Lesen von /dev/urandom");
            close(fd);
            return 1;
        }

    } while (random_value > max_acceptable);

    close(fd);

    // Gleichverteilte Zufallszahl zwischen 1 und 1000
    int result = (random_value % RANGE) + MIN;

    printf("Kryptografisch sichere Zufallszahl (1-1000): %d\n", result);
    return 0;
}
```

2.1 ZUFALLSZAHLEN

Wovon hängt die erzeugte Zufallszahl ab?

Die Zufallszahl, wie im Code generiert, stammt aus Daten, die von **`/dev/urandom`** gelesen wird. Die Ausgabe des Geräse (`/dev/urandom`) basiert auf folgenden Faktoren:

1) **Entropiequelle - Pool & Systemereignisse**

- Der Kernel sammelt kontinuierlich ENtropie aus Interrupt-Timings (z.B. Tastatur, Maus, Netzwerk)
- Fällt der Entropie-Pool unter die Hälfte seiner Kapazität, werden,neue Interrupt-Timings gesammelt, bis der Pool wieder gefüllt ist.

Entropie - bezeichnet den Grad an Zufälligkeit oder Unvorhersagbarkeit von Informationen.

Quelle: https://www.ibm.com/docs/de/aix/7.3.0?topic=files-random-urandom-devices

2.1 ZUFALLSZAHLEN

Wovon hängt die erzeugte Zufallszahl ab?

2) Cryptographically Secure Pseudo Random Number Generator (CSPRNG)-Verarbeitung

- /dev/urandom nutzt einen CSPRNG:
    - Verarbeitung der gesammelten Entropie
    - Bei unzureichender Frisch-Entropie wird zusätzlich kryptografischer Algorithmus verwendet, um weiter Ausgaben zu erzeugen

3) Modulo-freie-Prüfung, die Gleichverteilung sicherstellt

Quelle: https://www.ibm.com/docs/de/aix/7.3.0?topic=files-random-urandom-devices

2.2 E-MAIL LOGIN

Voraussetzung:

-OpenSSL (`libssl-dev`) installiert

-Ein Unix-System wegen `getpass`

```c
#include <stdio.h>          // Für printf, scanf, FILE usw.
#include <string.h>         // Für strcpy, strcmp, memset usw.
#include <stdlib.h>         // Für malloc, free, exit usw.
#include <openssl/evp.h>    // Für EVP-basiertes Hashing (PBKDF2)
#include <openssl/crypto.h> // Für sicheren Vergleich (CRYPTO_memcmp)
#include <unistd.h>   // Für getpass() (kein Echo bei Passwort)

#define SALT_LEN 16         // Länge des zufälligen Salt in Bytes
#define HASH_LEN 32         // Länge des SHA256-Hashes (256 Bit = 32 Byte)
#define ITERATIONS 100000   // Anzahl der PBKDF2-Iterationen
#define DB_FILE "users.db"  // Name der Datei, in der User gespeichert werden
#define MAX_USERS 100       // Maximale Anzahl an Benutzern

typedef struct {
    char email[100];                    // E-Mail des Benutzers
    unsigned char salt[SALT_LEN];       // Salt zur Passwortabsicherung
    unsigned char password_hash[HASH_LEN];  // Gehashter Passwortwert
} User;

//Sicherer Vergleich
int secure_compare(const unsigned char* a, const unsigned char* b, size_t len) {
    return CRYPTO_memcmp(a, b, len) == 0; //Timing-sicherer Vergleich
}
// Passwort mit PBKDF2 und SHA256 hashen
int hash_password(const char* password, const unsigned char* salt, unsigned char* hash) {
    return PKCS5_PBKDF2_HMAC(password, strlen(password), salt, SALT_LEN,
                             ITERATIONS, EVP_sha256(), HASH_LEN, hash);
}
// Benutzer aus Datei laden
int load_users(User* users, int* count) {
    FILE* f = fopen(DB_FILE, "rb");
    if (!f) return 0;
    *count = fread(users, sizeof(User), MAX_USERS, f);
    fclose(f);
    return 1;
}
// Benutzer anhand E-Mail suchen
int find_user(User* users, int count, const char* email) {
    for (int i = 0; i < count; i++) {
        if (strcmp(users[i].email, email) == 0)
            return i;
    }
    return -1;
}
// Login-Funktion
void login_user(User* users, int count) {
    char email[100];
    char* password;

    printf(" E-Mail: ");
    scanf("%99s", email);

    int index = find_user(users, count, email);
    if (index < 0) {
        printf(" Benutzer nicht gefunden.\n");
        return;
    }

    password = getpass("🔐 Passwort: ");
    unsigned char input_hash[HASH_LEN];

    if (!hash_password(password, users[index].salt, input_hash)) {
        printf("Fehler beim Hashen.\n");
        return;
    }

    if (secure_compare(users[index].password_hash, input_hash, HASH_LEN)) {
        printf("Login erfolgreich.\n");
    } else {
        printf("Falsches Passwort.\n");
    }
}

int main() {
    User users[MAX_USERS];
    int user_count = 0;

    if (!load_users(users, &user_count)) {
        printf(" Konnte Benutzerdaten nicht laden.\n");
        return 1;
    }

    login_user(users, user_count);
    return 0;
}
```

2.2 E-MAIL LOGIN

```c
#include <stdio.h>          // Für printf, scanf, FILE usw.
#include <string.h>         // Für strcpy, strcmp, memset usw.
#include <stdlib.h>         // Für malloc, free, exit usw.
#include <openssl/evp.h>    // Für EVP-basiertes Hashing (PBKDF2)
#include <openssl/crypto.h> // Für sicheren Vergleich (CRYPTO_memcmp)
#include <unistd.h>   // Für getpass() (kein Echo bei Passwort)

#define SALT_LEN 16         // Länge des zufälligen Salt in Bytes
#define HASH_LEN 32         // Länge des SHA256-Hashes (256 Bit = 32 Byte)
#define ITERATIONS 100000   // Anzahl der PBKDF2-Iterationen
#define DB_FILE "users.db"  // Name der Datei, in der User gespeichert
werden
#define MAX_USERS 100       // Maximale Anzahl an Benutzern

typedef struct {
   char email[100];                // E-Mail des Benutzers
   unsigned char salt[SALT_LEN];       // Salt zur Passwortabsicherung
   unsigned char password_hash[HASH_LEN];  // Gehashter Passwortwert
} User;

//Sicherer Vergleich
int secure_compare(const unsigned char* a, const unsigned char* b, size_t
len) {
   return CRYPTO_memcmp(a, b, len) == 0; //Timing-sicherer Vergleich
}
```

```c
// Passwort mit PBKDF2 und SHA256 hashen
int hash_password(const char* password, const unsigned char* salt, unsigned
char* hash) {
   return PKCS5_PBKDF2_HMAC(password, strlen(password), salt, SALT_LEN,
                            ITERATIONS, EVP_sha256(), HASH_LEN, hash);
}
// Benutzer aus Datei laden
int load_users(User* users, int* count) {
   FILE* f = fopen(DB_FILE, "rb");
   if (!f) return 0;
   *count = fread(users, sizeof(User), MAX_USERS, f);
   fclose(f);
   return 1;
}
// Benutzer anhand E-Mail suchen
int find_user(User* users, int count, const char* email) {
   for (int i = 0; i < count; i++) {
       if (strcmp(users[i].email, email) == 0)
           return i;
   }
   return -1;
}
```

2.2 E-MAIL LOGIN

```c
// Login-Funktion
void login_user(User* users, int count) {
    char email[100];
    char* password;

    printf(" E-Mail: ");
    scanf("%99s", email);

    int index = find_user(users, count, email);
    if (index < 0) {
        printf(" Benutzer nicht gefunden.\n");
        return;
    }

    password = getpass("🔐 Passwort: ");
    unsigned char input_hash[HASH_LEN];

    if (!hash_password(password, users[index].salt, input_hash)) {
        printf("Fehler beim Hashen.\n");
        return;
    }

    if (secure_compare(users[index].password_hash, input_hash, HASH_LEN)) {
        printf("Login erfolgreich.\n");
    } else {
        printf("Falsches Passwort.\n");
    }
}
```

```c
int main() {
    User users[MAX_USERS];
    int user_count = 0;

    if (!load_users(users, &user_count)) {
        printf(" Konnte Benutzerdaten nicht
laden.\n");
        return 1;
    }

    login_user(users, user_count);
    return 0;
}
```

3 REIFEGRADE

3.1.A SOFTWARE REQUIREMENTS

Strategie und den Umsetzungsplan, um Banking App Showcase den Maturity Level 2 gemäß SAMM v2 im Bereich Strategy & Metrics zu erreichen:

Ziel: Strategische Planung für Application Security mit Roadmap, Meilensteinen und Stakeholder-Alignment.

1.1 Maßnahmen

- Entwicklung einer AppSec-Roadmap für 1–3 Jahre basierend auf Risiken wie MITM, Session Hijacking, Datenverlust.
- Strategische Ziele definieren: TLS 1.3, 2FA, verschlüsselte Datenhaltung.
- Einbindung von IT, Compliance, Entwicklung und Legal.
- Veröffentlichung und Pflege der Roadmap auf interner Plattform

3.1.A SOFTWARE REQUIREMENTS

1.2 Ergebnis

- Strategie liegt vor und ist abgestimmt auf Risiken und Compliance-Vorgaben.
- Stakeholder-Buy-in gesichert.
- Transparente Roadmap etabliert.

3.1.B SUPPLIER SECURITY

Ziel: Sicherstellung, dass alle externen Lieferanten, Dienstleister oder Drittanbieter die Sicherheitsanforderungen der Organisation erfüllen.

- **Verträge enthalten verpflichtende Sicherheitsvorgaben:**
  - **Verschlüsselung**
  - **Zugriffsmanagement**
  - **Incident Response**
  - **Nachweise & Audits**